

FREQUENCY PRINCIPLE: FOURIER ANALYSIS SHEDS LIGHT ON DEEP NEURAL NETWORKS

Zhi-Qin John Xu*
Shanghai Jiao Tong University
xuzhiqin@sjtu.edu.cn

Yaoyu Zhang
Institute for Advanced Study
Yaoyu@ias.edu

Tao Luo
Purdue University
luo196@purdue.edu

Yanyang Xiao
Shenzhen Institutes of Advanced Technology
xyy82148@gmail.com

Zheng Ma
Purdue University
ma531@purdue.edu

ABSTRACT

We study the training process of Deep Neural Networks (DNNs) from the Fourier analysis perspective. We demonstrate a very universal Frequency Principle (F-Principle) — DNNs often fit target functions from low to high frequencies — on high-dimensional benchmark datasets such as MNIST/CIFAR10 and deep neural networks such as VGG16. This F-Principle of DNNs is opposite to the behavior of most conventional iterative numerical schemes (e.g., Jacobi method), which exhibit faster convergence for higher frequencies for various scientific computing problems. With a simple theory, we illustrate that this F-Principle results from the regularity of the commonly used activation functions. The F-Principle implies an implicit bias that DNNs tend to fit training data by a low-frequency function. This understanding provides an explanation of good generalization of DNNs on most real datasets and bad generalization of DNNs on parity function or randomized dataset.

1 INTRODUCTION

Understanding the training process of Deep Neural Networks (DNNs) is a fundamental problem in the area of deep learning. We find a common implicit bias in the gradient-based training process of DNNs, that is, a Frequency Principle (F-Principle):

DNNs often fit target functions from low to high frequencies during the training process.

In another word, at the early stage of training, the low-frequencies are fitted and as iteration steps of training increase, the high-frequencies are fitted. For example, when a DNN is trained to fit $y = \sin(x) + \sin(2x)$, its output would be close to $\sin(x)$ at early stage and as training goes on, its output would be close to $\sin(x) + \sin(2x)$. F-Principle was observed empirically in synthetic low-dimensional data with MSE loss during DNN training (Xu et al., 2018; Rahaman et al., 2018). However, in deep learning, empirical phenomena vary from one network structure to another, from one dataset to another and often show significant difference between synthetic data and high-dimensional real data. It is still of great challenges to quantitatively study the universality of empirical observed phenomena, e.g., F-Principle, in high-dimensional real problems due to large computational cost, for instance, the high-dimensional Fourier transform is prohibitive in practice. In addition, it is also unclear whether the F-Principle can guide the usage and provide insight of DNNs in real problems.

In this work, we first design two methods to show that the F-Principle exists in the training process of DNNs for different benchmark setups, e.g., MNIST (LeCun, 1998), CIFAR10 (Krizhevsky et al., 2010), and deep networks, such as VGG16 (Simonyan & Zisserman, 2014). The settings we have considered are i) different DNN architectures, e.g., fully-connected network and convolutional neural network (CNN); ii) different activation functions, e.g., tanh and rectified linear unit (ReLU); iii)

*Corresponding author, <https://ins.sjtu.edu.cn/people/xuzhiqin/>

different loss functions, e.g., cross entropy, mean squared error (MSE), and loss energy functional in variation problems. These results demonstrate the universality of the F-Principle.

To facilitate the designs and applications of DNN-based schemes, we characterize a stark difference between DNNs and conventional numerical schemes on various scientific computing problems, where most of the conventional methods (e.g., Jacobi method) exhibit the opposite convergence behavior — faster convergence for higher frequencies. This difference implicates that DNN can be adopted to accelerate the convergence of low frequencies for computational problems.

We also show how the power decaying spectrum of commonly used activation functions contributes to the F-Principle with a theory under an idealized setting. Note that this mechanism is rigorously demonstrated for DNNs of general settings in a subsequent work (Luo et al., 2019). Finally, we discuss that the F-Principle provides an understanding of good generalization of DNNs in many real datasets (Zhang et al., 2016) and poor generalization in learning the parity function (Shalev-Shwartz et al., 2017; Nye & Saxe, 2018), that is, the F-Principle which implies that DNNs prefer low frequencies, is consistent with the property of low frequencies dominance in many real datasets, e.g., MNIST/CIFAR10, but is different from the parity function whose spectrum concentrates on high frequencies. Compared with previous studies, which only study synthetic data with MSE loss, our main contributions are as follows:

1. By designing both the projection and filtering method, we consistently demonstrate the F-Principle for high-dimensional real datasets of MNIST/CIFAR10 over various architectures such as VGG16.
2. For the application of solving differential equations, we show (i) conventional numerical schemes learn higher frequencies faster whereas DNNs learn lower frequencies faster by the F-Principle, (ii) convergence of low frequencies can be greatly accelerated with DNN-based schemes.
3. We show a simple theory under an idealized setting for an easy understanding of the F-Principle.
4. We discuss in detail the impact of the F-Principle to the generalization of DNNs that DNNs are implicitly biased towards a low frequency function leading to good and poor generalization for low and high frequency dominant target functions respectively.

2 FREQUENCY PRINCIPLE

The concept of “frequency” is central to the understanding of F-Principle. In this paper, the “frequency” means **response frequency** NOT image (or input) frequency as explained in the following.

Image (or input) frequency (NOT used in the paper): Frequency of 2-d function $I : \mathbb{R}^2 \rightarrow \mathbb{R}$ representing the intensity of an image over pixels at different locations. This frequency corresponds to the rate of change of intensity *across neighbouring pixels*. For example, an image of constant intensity possesses only the zero frequency, i.e., the lowest frequency, while a sharp edge contributes to high frequencies of the image.

Response frequency (used in the paper): Frequency of a general Input-Output mapping f . For example, consider a simplified classification problem of partial MNIST data using only the data with label 0 and 1, $f(x_1, x_2, \dots, x_{784}) : \mathbb{R}^{784} \rightarrow \{0, 1\}$ mapping 784-d space of pixel values to 1-d space, where x_i is the intensity of the i -th pixel. Denote the mapping’s Fourier transform as $\hat{f}(k_1, k_2, \dots, k_{784})$. The frequency in the coordinate k_i measures the rate of change of $f(x_1, x_2, \dots, x_{784})$ with respect to x_i , i.e., the intensity of the i -th pixel. If f possesses significant high frequencies for large k_i , then a small change of x_i in the image might induce a large change of the output (e.g., adversarial example). For a dataset with multiple classes, we can similarly define frequency for each output dimension. An illustration of F-Principle using a function of 1-d input is in Appendix A.

Frequency Principle: DNNs often fit target functions from low to high (response) frequencies during the training process. In the following, by using high-dimensional real datasets, we experimentally demonstrate F-Principle at the levels of both individual frequencies (projection method) and coarse-grained frequencies (filtering method).

3 F-PRINCIPLE IN MNIST/CIFAR10 THROUGH PROJECTION METHOD

Real datasets are very different from synthetic data used in previous studies. In order to utilize the F-Principle to understand and better use DNNs in real datasets, it is important to verify whether the F-Principle also holds in high-dimensional real datasets.

In the following experiments, we examine the F-Principle in a training dataset of $\{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=0}^{n-1}$ where n is the size of dataset. $\mathbf{x}_i \in \mathbb{R}^d$ is a vector representing the image and $\mathbf{y}_i \in \{0, 1\}^{10}$ is the output (a one-hot vector indicating the label for the dataset of image classification). d is the dimension of the input ($d = 784$ for MNIST and $d = 32 \times 32 \times 3$ for CIFAR10). Since the high dimensional discrete Fourier transform (DFT) has high computational cost, in this section, we only consider one direction in the Fourier space through a projection method for each examination.

3.1 EXAMINATION METHOD: PROJECTION

For a dataset $\{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=0}^{n-1}$ we consider one entry of 10-d output, denoted by $y_i \in \mathbb{R}$. The high dimensional discrete non-uniform Fourier transform of $\{(\mathbf{x}_i, y_i)\}_{i=0}^{n-1}$ is $\hat{y}_{\mathbf{k}} = \frac{1}{n} \sum_{i=0}^{n-1} y_i \exp(-i2\pi \mathbf{k} \cdot \mathbf{x}_i)$. The number of all possible $\mathbf{k}$ grows exponentially on dimension d . For illustration, in each examination, we consider a direction of $\mathbf{k}$ in the Fourier space, i.e., $\mathbf{k} = k\mathbf{p}_1$, $\mathbf{p}_1$ is a chosen and fixed unit vector, hence $|\mathbf{k}| = k$. Then we have $\hat{y}_k = \frac{1}{n} \sum_{i=0}^{n-1} y_i \exp(-i2\pi(\mathbf{p}_1 \cdot \mathbf{x}_i)k)$, which is essentially the 1-d Fourier transform of $\{(x_{\mathbf{p}_1, i}, y_i)\}_{i=0}^{n-1}$, where $x_{\mathbf{p}_1, i} = \mathbf{p}_1 \cdot \mathbf{x}_i$ is the projection of $\mathbf{x}_i$ on the direction $\mathbf{p}_1$ (Bracewell & Bracewell, 1986). For each training dataset, $\mathbf{p}_1$ is chosen as the first principle component of the input space. To examine the convergence behavior of different frequency components during the training, we compute the relative difference between the DNN output and the target function for selected important frequencies k 's at each recording step, that is, $\Delta_F(k) = |\hat{h}_k - \hat{y}_k|/|\hat{y}_k|$, where $\hat{y}_k$ and $\hat{h}_k$ are 1-d Fourier transforms of $\{y_i\}_{i=0}^{n-1}$ and the corresponding DNN output $\{h_i\}_{i=0}^{n-1}$, respectively, along $\mathbf{p}_1$.

3.2 MNIST/CIFAR10

In the following, we show empirically that the F-Principle is exhibited in the selected direction during the training process of DNNs when applied to MNIST/CIFAR10 with cross-entropy loss. The network for MNIST is a fully-connected tanh DNN (784-400-200-10) and for CIFAR10 is two ReLU convolutional layers followed by a fully-connected DNN (800-400-400-400-10). All experimental details of this paper can be found in Appendix B. We consider one of the 10-d outputs in each case using non-uniform Fourier transform, as shown in Fig. 1(a) and 1(c), low frequencies dominate in both real datasets. During the training, the evolution of relative errors of certain selected frequencies (marked by black squares in Fig. 1(a) and 1(c)) is shown in Fig. 1(b) and 1(d). One can easily observe that DNNs capture low frequencies first and gradually capture higher frequencies. Clearly, this behavior is consistent with the F-Principle. For other components of the output vector and other directions of $\mathbf{p}$, similar phenomena are also observed.

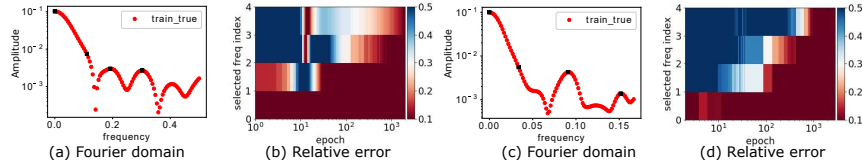


Figure 1: Projection method. (a, b) are for MNIST, (c, d) for CIFAR10. (a, c) Amplitude $|\hat{y}_k|$ vs. frequency. Selected frequencies are marked by black squares. (b, d) $\Delta_F(k)$ vs. training epochs for the selected frequencies.

4 F-PRINCIPLE IN MNIST/CIFAR10 THROUGH FILTERING METHOD

The projection method in previous section enables us to visualize the F-Principle in one direction for each examination at the level of individual frequency components. However, this demonstration of the F-Principle is insufficient since it is impossible to verify F-Principle at all potentially informative directions for high-dimensional data. To compensate the projection method, in this section, we consider a coarse-grained filtering method which is able to unravel whether, in the radially averaged sense, low frequencies converge faster than high frequencies.

4.1 EXAMINATION METHOD: FILTERING

We split the frequency domain into two parts, i.e., a low-frequency part with $|\mathbf{k}| \leq k_0$ and a high-frequency part with $|\mathbf{k}| > k_0$, where $|\cdot|$ is the length of a vector. The DNN is trained as usual by the original dataset $\{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=0}^{n-1}$, such as MNIST or CIFAR10. The DNN output is denoted as $\mathbf{h}$. During the training, we can examine the convergence of relative errors of low- and high- frequency part, using the two measures below

$$e_{\text{low}} = \left(\frac{\sum_{\mathbf{k}} \mathbb{1}_{|\mathbf{k}| \leq k_0} |\hat{\mathbf{y}}(\mathbf{k}) - \hat{\mathbf{h}}(\mathbf{k})|^2}{\sum_{\mathbf{k}} \mathbb{1}_{|\mathbf{k}| \leq k_0} |\hat{\mathbf{y}}(\mathbf{k})|^2} \right)^{\frac{1}{2}}, \quad e_{\text{high}} = \left(\frac{\sum_{\mathbf{k}} (1 - \mathbb{1}_{|\mathbf{k}| \leq k_0}) |\hat{\mathbf{y}}(\mathbf{k}) - \hat{\mathbf{h}}(\mathbf{k})|^2}{\sum_{\mathbf{k}} (1 - \mathbb{1}_{|\mathbf{k}| \leq k_0}) |\hat{\mathbf{y}}(\mathbf{k})|^2} \right)^{\frac{1}{2}},$$

respectively, where $\hat{\cdot}$ indicates Fourier transform, $\mathbb{1}_{|\mathbf{k}| \leq k_0}$ is an indicator function, i.e.,

$$\mathbb{1}_{|\mathbf{k}| \leq k_0} = \begin{cases} 1, & |\mathbf{k}| \leq k_0, \\ 0, & |\mathbf{k}| > k_0. \end{cases}$$

If we consistently observe $e_{\text{low}} < e_{\text{high}}$ for different k_0 's during the training, then in a mean sense, lower frequencies are first captured by the DNN, i.e., F-Principle.

However, because it is almost impossible to compute above quantities numerically due to high computational cost of high-dimensional Fourier transform, we alternatively use the Fourier transform of a Gaussian function $\hat{G}^\delta(\mathbf{k})$, where δ is the variance of the Gaussian function G , to approximate $\mathbb{1}_{|\mathbf{k}| > k_0}$. This is reasonable due to the following two reasons. First, the Fourier transform of a Gaussian is still a Gaussian, i.e., $\hat{G}^\delta(\mathbf{k})$ decays exponentially as $|\mathbf{k}|$ increases, therefore, it can approximate $\mathbb{1}_{|\mathbf{k}| \leq k_0}$ by $\hat{G}^\delta(\mathbf{k})$ with a proper $\delta(k_0)$ (referred to as δ for simplicity). Second, the computation of e_{low} and e_{high} contains the multiplication of Fourier transforms in the frequency domain, which is equivalent to the Fourier transform of a convolution in the spatial domain. We can equivalently perform the examination in the spatial domain so as to avoid the almost impossible high-dimensional Fourier transform. The low frequency part can be derived by

$$\mathbf{y}_i^{\text{low}, \delta} \triangleq (\mathbf{y} * G^\delta)_i, \quad (1)$$

where $*$ indicates convolution operator, and the high frequency part can be derived by

$$\mathbf{y}_i^{\text{high}, \delta} \triangleq \mathbf{y}_i - \mathbf{y}_i^{\text{low}, \delta}. \quad (2)$$

Then, we can examine

$$e_{\text{low}} = \left(\frac{\sum_i |\mathbf{y}_i^{\text{low}, \delta} - \mathbf{h}_i^{\text{low}, \delta}|^2}{\sum_i |\mathbf{y}_i^{\text{low}, \delta}|^2} \right)^{\frac{1}{2}}, \quad e_{\text{high}} = \left(\frac{\sum_i |\mathbf{y}_i^{\text{high}, \delta} - \mathbf{h}_i^{\text{high}, \delta}|^2}{\sum_i |\mathbf{y}_i^{\text{high}, \delta}|^2} \right)^{\frac{1}{2}}, \quad (3)$$

where $\mathbf{h}_i^{\text{low}, \delta}$ and $\mathbf{h}_i^{\text{high}, \delta}$ are obtained from the DNN output $\mathbf{h}$ through the same decomposition. If $e_{\text{low}} < e_{\text{high}}$ for different δ 's during the training, F-Principle holds; otherwise, it is falsified. Next, we introduce the experimental procedure.

Step One: Training. Train the DNN by the *original dataset* $\{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=0}^{n-1}$, such as MNIST or CIFAR10. $\mathbf{x}_i$ is an image vector, $\mathbf{y}_i$ is a one-hot vector.

Step Two: Filtering. The low frequency part can be derived by

$$\mathbf{y}_i^{\text{low}, \delta} = \frac{1}{C_i} \sum_{j=0}^{n-1} \mathbf{y}_j G^\delta(\mathbf{x}_i - \mathbf{x}_j), \quad (4)$$

where $C_i = \sum_{j=0}^{n-1} G^\delta(\mathbf{x}_i - \mathbf{x}_j)$ is a normalization factor and

$$G^\delta(\mathbf{x}_i - \mathbf{x}_j) = \exp(-|\mathbf{x}_i - \mathbf{x}_j|^2 / (2\delta)). \quad (5)$$

The high frequency part can be derived by $\mathbf{y}_i^{\text{high},\delta} \triangleq \mathbf{y}_i - \mathbf{y}_i^{\text{low},\delta}$. We also compute $\mathbf{h}_i^{\text{low},\delta}$ and $\mathbf{h}_i^{\text{high},\delta}$ for each DNN output $\mathbf{h}_i$.

Step Three: Examination. To quantify the convergence of $\mathbf{h}^{\text{low},\delta}$ and $\mathbf{h}^{\text{high},\delta}$, we compute the relative error e_{low} and e_{high} through Eq. (3).

4.2 DNNs WITH VARIOUS SETTINGS

It is important to verify the F-Principle in commonly used large networks. With the filtering method, we show the F-Principle in the DNN training process of real datasets. For MNIST, we use a fully-connected tanh-DNN (no softmax) with MSE loss; for CIFAR10, we use cross-entropy loss and two structures, one is small ReLU-CNN network, i.e., two convolutional layers, followed by a fully-connected multi-layer neural network with a softmax; the other is VGG16 (Simonyan & Zisserman, 2014) equipped with a 1024 fully-connected layer. These three structures are denoted as “DNN”, “CNN” and “VGG” in Fig. 2, respectively. All are trained by SGD from *scratch*. More details are in Appendix B

We scan a large range of δ for both datasets, as an example, results of each dataset for several δ ’s are shown in Fig. 2, respectively. Red color indicates small relative error. In all cases, the relative error of the low-frequency part, i.e., e_{low} , decreases (turns red) much faster than that of the high-frequency part, i.e., e_{high} . As analyzed above, the low-frequency part converges faster than the high-frequency part. We also remark that, based on above the results on cross-entropy loss, the F-Principle is not limited to MSE loss, which possesses a natural Fourier domain interpretation by the Parseval’s theorem. Note that the above results holds for both SGD and GD.

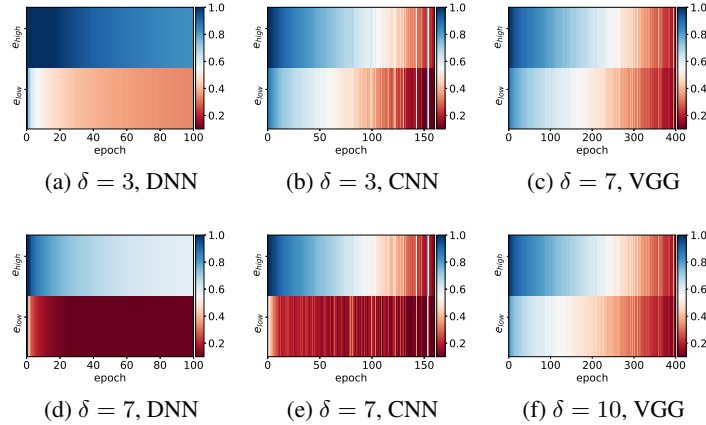


Figure 2: F-Principle in real datasets. e_{low} and e_{high} indicated by color against training epoch.

5 F-PRINCIPLE IN SOLVING DIFFERENTIAL EQUATION

Recently, DNN-based approaches have been actively explored for a variety of scientific computing problems, e.g., solving high-dimensional partial differential equations (E et al., 2017; Khoo et al., 2017; He et al., 2018; Fan et al., 2018) and molecular dynamics (MD) simulations (Han et al., 2017). However, the behaviors of DNNs applied to these problems are not well-understood. To facilitate the designs and applications of DNN-based schemes, it is important to characterize the difference between DNNs and conventional numerical schemes on various scientific computing problems. In this section, focusing on solving Poisson’s equation, which has broad applications in mechanical

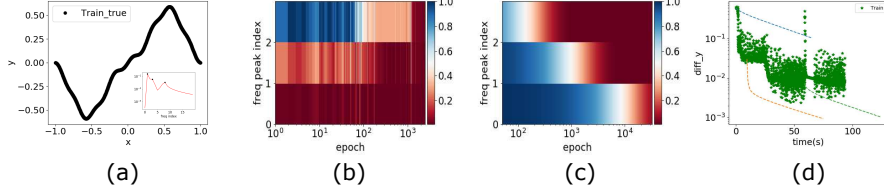


Figure 3: Poisson’s equation. (a) $u_{\text{ref}}(x)$. Inset: $|\hat{u}_{\text{ref}}(k)|$ as a function of frequency. Frequencies peaks are marked with black dots. (b,c) $\Delta_F(k)$ computed on the inputs of training data at different epochs for the selected frequencies for DNN (b) and Jacobi (c). (d) $|h - u_{\text{ref}}|_{\infty}$ at different running time. Green stars indicate $|h - u_{\text{ref}}|_{\infty}$ using DNN alone. The dashed lines indicate $|h - u_{\text{ref}}|_{\infty}$ for the Jacobi method with different colors indicating initialization by different timing of DNN training.

engineering and theoretical physics (Evans, 2010), we highlight a stark difference of a DNN-based solver and the Jacobi method during the training/iteration, which can be explained by the F-Principle.

Consider a 1-d Poisson’s equation:

$$-\Delta u(x) = g(x), \quad x \in \Omega \triangleq (-1, 1), \quad (6)$$

$$u(-1) = u(1) = 0. \quad (7)$$

We consider the example with $g(x) = \sin(x) + 4\sin(4x) - 8\sin(8x) + 16\sin(24x)$ which has analytic solution $u_{\text{ref}}(x) = g_0(x) + c_1x + c_0$, where $g_0 = \sin(x) + \sin(4x)/4 - \sin(8x)/8 + \sin(24x)/36$, $c_1 = (g_0(-1) - g_0(1))/2$ and $c_0 = -(g_0(-1) + g_0(1))/2$. 1001 training samples $\{x_i\}_{i=0}^n$ are evenly spaced with grid size δx in $[0, 1]$. Here, we use the DNN output, $h(x; \theta)$, to fit $u_{\text{ref}}(x)$ (Fig. 3(a)). A DNN-based scheme is proposed by considering the following empirical loss function (E & Yu, 2018),

$$I_{\text{emp}} = \sum_{i=1}^{n-1} \left(\frac{1}{2} |\nabla_x h(x_i)|^2 - g(x_i)h(x_i) \right) \delta x + \beta (h(x_0)^2 + h(x_n)^2). \quad (8)$$

The second term in $I_{\text{emp}}(h)$ is a penalty, with constant β , arising from the Dirichlet boundary condition (7). After training, the DNN output well matches the analytical solution u_{ref} . Focusing on the convergence of three peaks (inset of Fig. 3(a)) in the Fourier transform of u_{ref} , as shown in Fig. 3(b), low frequencies converge faster than high frequencies as predicted by the F-Principle. For comparison, we also use the Jacobi method to solve problem (6). High frequencies converge faster in the Jacobi method (Details can be found in Appendix C), as shown in Fig. 3(c).

As a demonstration, we further propose that DNN can be combined with conventional numerical schemes to accelerate the convergence of low frequencies for computational problems. First, we solve the Poisson’s equation in Eq. (6) by DNN with M optimization steps (or epochs), which needs to be chosen carefully, to get a good initial guess in the sense that this solution has already learned the low frequencies (large eigenvalues) part. Then, we use the Jacobi method with the new initial data for the further iterations. We use $|h - u_{\text{ref}}|_{\infty} \triangleq \max_{x \in \Omega} |h(x) - u_{\text{ref}}(x)|$ to quantify the learning result. As shown by green stars in Fig. 3(d), $|h - u_{\text{ref}}|_{\infty}$ fluctuates after some running time using DNN only. Dashed lines indicate the evolution of the Jacobi method with initial data set to the DNN output at the corresponding steps. If M is too small (stop too early) (left dashed line), which is equivalent to only using Jacobi, it would take long time to converge to a small error, because low frequencies converges slowly, yet. If M is too big (stop too late) (right dashed line), which is equivalent to using DNN only, much time would be wasted for the slow convergence of high frequencies. A proper choice of M is indicated by the initial point of orange dashed line, in which low frequencies are quickly captured by the DNN, followed by fast convergence in high frequencies of the Jacobi method.

This example illustrates a cautionary tale that, although DNNs has clear advantage, using DNNs alone may not be the best option because of its limitation of slow convergence at high frequencies. Taking advantage of both DNNs and conventional methods to design faster schemes could be a promising direction in scientific computing problems.

6 A PRELIMINARY THEORETICAL UNDERSTANDING

A subsequent theoretical work (Luo et al., 2019) provides a rigorous mathematical study of the F-Principle at different frequencies for general DNNs (e.g., multiple hidden layers, different activation functions, high-dimensional inputs). The key insight is that the regularity of DNN converts into the decay rate of a loss function in the frequency domain. For an intuitive understanding of this key insight, we present a simplified theory, which connects the smoothness of the activation function with different gradient priorities in frequency domain. Although this naive theory is in an ideal setting, it is much easier for understanding.

The activation function we consider is $\sigma(x) = \tanh(x)$, which is smooth in spatial domain and its derivative decays exponentially with respect to frequency in the Fourier domain. For a DNN of one hidden layer with m nodes, 1-d input x and 1-d output: $h(x) = \sum_{j=1}^m a_j \sigma(w_j x + b_j)$, $a_j, w_j, b_j \in \mathbb{R}$. We also use the notation $\theta = \{\theta_{lj}\}$ with $\theta_{1j} = a_j$, $\theta_{2j} = w_j$, and $\theta_{3j} = b_j$, $j = 1, \dots, m$. The loss at frequency k is $L(k) = \frac{1}{2} \left| \hat{h}(k) - \hat{f}(k) \right|^2$, $\hat{\cdot}$ is the Fourier transform, f is the target function.

The total loss function is defined as: $L = \int_{-\infty}^{+\infty} L(k) dk$. Note that according to the Parseval's theorem, this loss function in the Fourier domain is equal to the commonly used MSE loss. We have the following theorems (The proofs are at Appendix D.). Define $W = (w_1, w_2, \dots, w_m)^T \in \mathbb{R}^m$.

Theorem 1. *Considering a DNN of one hidden layer with activation function $\sigma(x) = \tanh(x)$, for any frequencies k_1 and k_2 such that $|\hat{f}(k_1)| > 0$, $|\hat{f}(k_2)| > 0$, and $|k_2| > |k_1| > 0$, there exist positive constants c and C such that for sufficiently small δ , we have*

$$\frac{\mu \left(\left\{ W : \left| \frac{\partial L(k_1)}{\partial \theta_{lj}} \right| > \left| \frac{\partial L(k_2)}{\partial \theta_{lj}} \right| \text{ for all } l, j \right\} \cap B_\delta \right)}{\mu(B_\delta)} \geq 1 - C \exp(-c/\delta),$$

where $B_\delta \subset \mathbb{R}^m$ is a ball with radius δ centered at the origin and $\mu(\cdot)$ is the Lebesgue measure.

Theorem 1 indicates that for any two non-converged frequencies, with small weights, the lower-frequency gradient exponentially dominates over the higher-frequency ones. Due to the Parseval's theorem, the MSE loss in the spatial domain is equivalent to the L2 loss in the Fourier domain. To intuitively understand the higher decay rate of a lower-frequency loss function, we consider the training in the Fourier domain with loss function of only two non-zero frequencies.

Theorem 2. *Considering a DNN of one hidden layer with activation function $\sigma(x) = \tanh(x)$. Suppose the target function has only two non-zero frequencies k_1 and k_2 , that is, $|\hat{f}(k_1)| > 0$, $|\hat{f}(k_2)| > 0$, $|k_2| > |k_1| > 0$, and $|\hat{f}(k)| = 0$ for $k \neq k_1, k_2$. Consider the loss function of $L = L(k_1) + L(k_2)$ with gradient descent training. Denote*

$$\mathcal{S} = \left\{ \frac{\partial L(k_1)}{\partial t} \leq 0, \frac{\partial L(k_1)}{\partial t} \leq \frac{\partial L(k_2)}{\partial t} \right\},$$

that is, $L(k_1)$ decreases faster than $L(k_2)$. There exist positive constants c and C such that for sufficiently small δ , we have

$$\frac{\mu(\{W : \mathcal{S} \text{ holds}\} \cap B_\delta)}{\mu(B_\delta)} \geq 1 - C \exp(-c/\delta),$$

where $B_\delta \subset \mathbb{R}^m$ is a ball with radius δ centered at the origin and $\mu(\cdot)$ is the Lebesgue measure.

7 DISCUSSIONS

Next, we discuss DNN's generalization ability from the view point of Fourier analysis.

DNNs often generalize well in real problems (Zhang et al., 2016) but badly in fitting the parity function (Shalev-Shwartz et al., 2017; Nye & Saxe, 2018). Understanding the differences between above two types of problems, i.e., good and bad generalization performance of DNN, is critical. Next, we show a qualitative difference between these two types of problems through *Fourier analysis* and use the *F-Principle* to provide insight into how different characteristics in Fourier domain result in different generalization performances of DNNs.

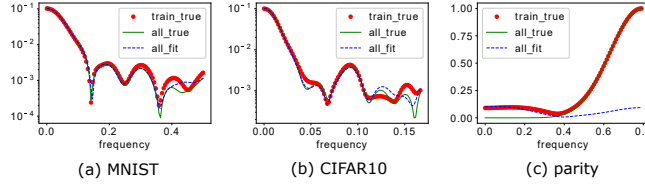


Figure 4: Fourier analysis for different generalization ability. The plot is the amplitude of the Fourier coefficient against frequency k . The red dots are for the training dataset, the green line is for the whole dataset, and the blue dashed line is for an output of well-trained DNN on the input of the whole dataset. For (c), $d = 10$. The training data is 200 randomly selected points.

For MNIST/CIFAR10, we examined $\hat{y}_{\text{total},\mathbf{k}} = \frac{1}{n_{\text{total}}} \sum_{i=0}^{n_{\text{total}}-1} y_i \exp(-i2\pi\mathbf{k} \cdot \mathbf{x}_i)$, where $\{(\mathbf{x}_i, y_i)\}_{i=0}^{n_{\text{total}}-1}$ consists of both the training and test datasets with certain selected output component, at different directions of $\mathbf{k}$ in the Fourier space. We find that $\hat{y}_{\text{total},\mathbf{k}}$ concentrates on the low frequencies along those examined directions. For illustration, $\hat{y}_{\text{total},\mathbf{k}}$'s along the first principle component are shown by green lines in Fig. 4(a, b) for MNIST/CIFAR10, respectively. When only the training dataset is used, $\hat{y}_{\text{train},\mathbf{k}}$ well overlaps with $\hat{y}_{\text{total},\mathbf{k}}$ at the dominant low frequencies.

For the parity function $f(\mathbf{x}) = \prod_{j=1}^d x_j$ defined on $\Omega = \{-1, 1\}^d$, its Fourier transform is $\hat{f}(\mathbf{k}) = \frac{1}{2^d} \sum_{\mathbf{x} \in \Omega} \prod_{j=1}^d x_j e^{-i2\pi\mathbf{k} \cdot \mathbf{x}} = (-i)^d \prod_{j=1}^d \sin 2\pi k_j$. Clearly, for $\mathbf{k} \in [-\frac{1}{4}, \frac{1}{4}]^d$, the power of the parity function concentrates at $\mathbf{k} \in \{-\frac{1}{4}, \frac{1}{4}\}^d$ and vanishes as $\mathbf{k} \rightarrow \mathbf{0}$, as illustrated in Fig. 4(c) for the direction of $\mathbf{1}_d$. Given a randomly sampled training dataset $S \subset \Omega$ with s points, the nonuniform Fourier transform on S is computed as $\hat{f}_S(\mathbf{k}) = \frac{1}{s} \sum_{\mathbf{x} \in S} \prod_{j=1}^d x_j e^{-i2\pi\mathbf{k} \cdot \mathbf{x}}$. As shown in Fig. 4(c), $\hat{f}(\mathbf{k})$ and $\hat{f}_S(\mathbf{k})$ significantly differ at low frequencies.

By experiments, the generalization ability of DNNs can be well reflected by the Fourier analysis. For the MNIST/CIFAR10, we observed the Fourier transform of the output of a well-trained DNN on $\{\mathbf{x}_i\}_{i=0}^{n_{\text{total}}-1}$ faithfully recovers the dominant low frequencies, as illustrated in Fig. 4(a) and 4(b), respectively, indicating a good generalization performance as observed in experiments. However, for the parity function, we observed that the Fourier transform of the output of a well-trained DNN on $\{\mathbf{x}_i\}_{i \in S}$ significantly deviates from $\hat{f}(\mathbf{k})$ at almost all frequencies, as illustrated in Fig. 4(c), indicating a bad generalization performance as observed in experiments.

The F-Principle implicates that among all the functions that can fit the training data, a DNN is implicitly biased during the training towards a function with more power at low frequencies. If the target function has significant high-frequency components, insufficient training samples will lead to artificial low frequencies in training dataset, such as the parity function as shown in Fig. 4(c), which is the well-known *aliasing* effect. Based on the F-Principle, as demonstrated in Fig. 4(c), these artificial low frequency components will be first captured to explain the training samples, whereas the high frequency components will be compromised by DNN. For MNIST/CIFAR10, since the power of high frequencies is much smaller than that of low frequencies, artificial low frequencies caused by aliasing can be neglected. To conclude, the distribution of power in Fourier domain of above two types of problems exhibits significant differences, which result in different generalization performances of DNNs according to the F-Principle.

8 RELATED WORK

As widely observed in experiments, DNNs with gradient-based training show different generalization abilities for different problems. On the one hand, there are different approaches attempting to explain why the training process often leads to a DNN of good generalization ability even when the number of parameters is much larger than the number of training data (Zhang et al., 2016). For example, generalization error is related to various complexity measures (Bartlett et al., 1999; Bartlett & Mendelson, 2002; Bartlett et al., 2017a;b; Neyshabur et al., 2017; Golowich et al., 2017; Dziugaite

& Roy, 2017; Neyshabur et al., 2018; E et al., 2018), local properties (sharpness/flatness) of loss functions at minima (Hochreiter & Schmidhuber, 1995; Keskar et al., 2016; Dinh et al., 2017; Wu et al., 2017), stability of optimization algorithms (Bousquet & Elisseeff, 2002; Xu & Mannor, 2012; Hardt et al., 2015), and implicit bias of the training process (Neyshabur et al., 2014; Poggio et al., 2018; Soudry et al., 2018; Arpit et al., 2017; Xu et al., 2018). On the other hand, several works focus on the failure of DNNs (Shalev-Shwartz et al., 2017; Nye & Saxe, 2018), e.g., fitting the parity function, in which a well-trained DNN possesses no generalization ability.

In the revised version, Rahaman et al. (2018) also examines the F-Principle in the MNIST dataset. However, they add noise to MNIST, which contaminates the labels. They only examine not very deep (6-layer) fully connected ReLU network with MSE loss, while cross-entropy loss is widely used.

ACKNOWLEDGMENTS

The authors want to thank Weinan E (Princeton University) and David W. McLaughlin (New York University) for helpful discussions and thank Yang Yuan (Tsinghua University) and Zhanxing Zhu (Peking University) for critically reading the manuscript. Part of this work was done when ZX, YZ, YX are postdocs at New York University Abu Dhabi and visiting members at Courant Institute supported by the NYU Abu Dhabi Institute G1301. The authors declare no conflict of interest.

REFERENCES

- Devansh Arpit, Stanislaw Jastrzebski, Nicolas Ballas, David Krueger, Emmanuel Bengio, Maxinder S Kanwal, Tegan Maharaj, Asja Fischer, Aaron Courville, Yoshua Bengio, et al. A closer look at memorization in deep networks. *arXiv preprint arXiv:1706.05394*, 2017. 8
- Peter L Bartlett and Shahar Mendelson. Rademacher and gaussian complexities: Risk bounds and structural results. *Journal of Machine Learning Research*, 3(Nov):463–482, 2002. 8
- Peter L Bartlett, Vitaly Maiorov, and Ron Meir. Almost linear vc dimension bounds for piecewise polynomial networks. In *Advances in Neural Information Processing Systems*, pp. 190–196, 1999. 8
- Peter L Bartlett, Dylan J Foster, and Matus J Telgarsky. Spectrally-normalized margin bounds for neural networks. In *Advances in Neural Information Processing Systems*, pp. 6240–6249, 2017a. 8
- Peter L Bartlett, Nick Harvey, Chris Liaw, and Abbas Mehrabian. Nearly-tight vc-dimension and pseudodimension bounds for piecewise linear neural networks. *arXiv preprint arXiv:1703.02930*, 2017b. 8
- Olivier Bousquet and André Elisseeff. Stability and generalization. *Journal of machine learning research*, 2(Mar):499–526, 2002. 8
- Ronald Newbold Bracewell and Ronald N Bracewell. *The Fourier transform and its applications*, volume 31999. McGraw-Hill New York, 1986. 3.1
- Laurent Dinh, Razvan Pascanu, Samy Bengio, and Yoshua Bengio. Sharp minima can generalize for deep nets. *arXiv preprint arXiv:1703.04933*, 2017. 8
- Gintare Karolina Dziugaite and Daniel M Roy. Computing nonvacuous generalization bounds for deep (stochastic) neural networks with many more parameters than training data. *arXiv preprint arXiv:1703.11008*, 2017. 8
- Weinan E and Bing Yu. The deep ritz method: A deep learning-based numerical algorithm for solving variational problems. *Communications in Mathematics and Statistics*, 6(1):1–12, 2018. 5
- Weinan E, Jiequn Han, and Arnulf Jentzen. Deep learning-based numerical methods for high-dimensional parabolic partial differential equations and backward stochastic differential equations. *Communications in Mathematics and Statistics*, 5(4):349–380, 2017. 5
- Weinan E, Chao Ma, and Lei Wu. A priori estimates of the generalization error for two-layer neural networks. *arXiv preprint arXiv:1810.06397*, 2018. 8

- Lawrence C Evans. Partial differential equations. 2010. 5
- Yuwei Fan, Lin Lin, Lexing Ying, and Leonardo Zepeda-Núñez. A multiscale neural network based on hierarchical matrices. *arXiv preprint arXiv:1807.01883*, 2018. 5
- Noah Golowich, Alexander Rakhlin, and Ohad Shamir. Size-independent sample complexity of neural networks. *arXiv preprint arXiv:1712.06541*, 2017. 8
- Jiequn Han, Linfeng Zhang, Roberto Car, et al. Deep potential: A general representation of a many-body potential energy surface. *arXiv preprint arXiv:1707.01478*, 2017. 5
- Moritz Hardt, Benjamin Recht, and Yoram Singer. Train faster, generalize better: Stability of stochastic gradient descent. *arXiv preprint arXiv:1509.01240*, 2015. 8
- Juncai He, Lin Li, Jinchao Xu, and Chunyue Zheng. Relu deep neural networks and linear finite elements. *arXiv preprint arXiv:1807.03973*, 2018. 5
- Sepp Hochreiter and Jürgen Schmidhuber. Simplifying neural nets by discovering flat minima. In *Advances in neural information processing systems*, pp. 529–536, 1995. 8
- Nitish Shirish Keskar, Dheevatsa Mudigere, Jorge Nocedal, Mikhail Smelyanskiy, and Ping Tak Peter Tang. On large-batch training for deep learning: Generalization gap and sharp minima. *arXiv preprint arXiv:1609.04836*, 2016. 8
- Yuehaw Khoo, Jianfeng Lu, and Lexing Ying. Solving parametric pde problems with artificial neural networks. *arXiv preprint arXiv:1707.03351*, 2017. 5
- Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014. B, 6
- Alex Krizhevsky, Vinod Nair, and Geoffrey Hinton. Cifar-10 (canadian institute for advanced research). URL <http://www.cs.toronto.edu/kriz/cifar.html>, 2010. 1
- Yann LeCun. The mnist database of handwritten digits. <http://yann.lecun.com/exdb/mnist/>, 1998. 1
- Tao Luo, Zheng Ma, Zhi-Qin John Xu, and Yaoyu Zhang. Theory of the frequency principle for general deep neural networks. *arXiv preprint arXiv:1906.09235*, 2019. 1, 6
- Behnam Neyshabur, Ryota Tomioka, and Nathan Srebro. In search of the real inductive bias: On the role of implicit regularization in deep learning. *arXiv preprint arXiv:1412.6614*, 2014. 8
- Behnam Neyshabur, Srinadh Bhojanapalli, David McAllester, and Nati Srebro. Exploring generalization in deep learning. In *Advances in Neural Information Processing Systems*, pp. 5947–5956, 2017. 8
- Behnam Neyshabur, Zhiyuan Li, Srinadh Bhojanapalli, Yann LeCun, and Nathan Srebro. Towards understanding the role of over-parametrization in generalization of neural networks. *arXiv preprint arXiv:1805.12076*, 2018. 8
- Maxwell Nye and Andrew Saxe. Are efficient deep representations learnable? 2018. 1, 7, 8
- T Poggio, K Kawaguchi, Q Liao, B Miranda, L Rosasco, X Boix, J Hidary, and HN Mhaskar. Theory of deep learning iii: the non-overfitting puzzle. Technical report, Technical report, CBMM memo 073, 2018. 8
- Nasim Rahaman, Devansh Arpit, Aristide Baratin, Felix Draxler, Min Lin, Fred A Hamprecht, Yoshua Bengio, and Aaron Courville. On the spectral bias of deep neural networks. *arXiv preprint arXiv:1806.08734*, 2018. 1, 8, A, F.2
- Shai Shalev-Shwartz, Ohad Shamir, and Shaked Shammah. Failures of gradient-based deep learning. *arXiv preprint arXiv:1703.07950*, 2017. 1, 7, 8
- Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. *arXiv preprint arXiv:1409.1556*, 2014. 1, 4.2, F.3

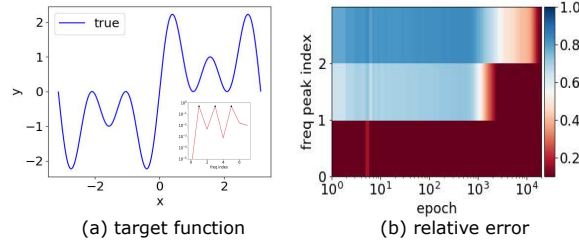


Figure 5: 1d input. (a) $f(x)$. Inset : $|\hat{f}(k)|$. (b) $\Delta_F(k)$ of three important frequencies (indicated by black dots in the inset of (a)) against different training epochs.

Daniel Soudry, Elad Hoffer, Mor Shpigel Nacson, Suriya Gunasekar, and Nathan Srebro. The implicit bias of gradient descent on separable data. *Journal of Machine Learning Research*, 19(70), 2018. 8

Lei Wu, Zhanxing Zhu, and Weinan E. Towards understanding generalization of deep learning: Perspective of loss landscapes. *arXiv preprint arXiv:1706.10239*, 2017. 8

Huan Xu and Shie Mannor. Robustness and generalization. *Machine learning*, 86(3):391–423, 2012. 8

Zhi-Qin J Xu, Yaoyu Zhang, and Yanyang Xiao. Training behavior of deep neural network in frequency domain. *arXiv preprint arXiv:1807.01251*, 2018. 1, 8, A, F.2

Chiyuan Zhang, Samy Bengio, Moritz Hardt, Benjamin Recht, and Oriol Vinyals. Understanding deep learning requires rethinking generalization. *arXiv preprint arXiv:1611.03530*, 2016. 1, 7, 8

A ILLUSTRATION OF F-PRINCIPLE FOR 1-D SYNTHETIC DATA

To illustrate the phenomenon of F-Principle, we use 1-d synthetic data to show the evolution of relative training error at different frequencies during the training of DNN. we train a DNN to fit a 1-d target function $f(x) = \sin(x) + \sin(3x) + \sin(5x)$ of three frequency components. On $n = 201$ evenly spaced training samples, i.e., $\{x_i\}_{i=0}^{n-1}$ in $[-3.14, 3.14]$, the discrete Fourier transform (DFT) of $f(x)$ or the DNN output (denoted by $h(x)$) is computed by $\hat{f}_k = \frac{1}{n} \sum_{i=0}^{n-1} f(x_i) e^{-i2\pi ik/n}$ and $\hat{h}_k = \frac{1}{n} \sum_{i=0}^{n-1} h(x_i) e^{-i2\pi ik/n}$, where k is the frequency. As shown in Fig. 5(a), the target function has three important frequencies as we design (black dots at the inset in Fig. 5(a)). To examine the convergence behavior of different frequency components during the training with MSE, we compute the relative difference of the DNN output and the target function for the three important frequencies k 's at each recording step, that is, $\Delta_F(k) = |\hat{h}_k - \hat{f}_k|/|\hat{f}_k|$, where $|\cdot|$ denotes the norm of a complex number. As shown in Fig. 5(b), the DNN converges the first frequency peak very fast, while converging the second frequency peak much slower, followed by the third frequency peak.

Next, we investigate the F-Principle on real datasets with more general loss functions other than MSE which was the only loss studied in the previous works (Xu et al., 2018; Rahaman et al., 2018). All experimental details can be found in Appendix. B.

B EXPERIMENTAL SETTINGS

In Fig. 5, the parameters of the DNN is initialized by a Gaussian distribution with mean 0 and standard deviation 0.1. We use a tanh-DNN with widths 1-8000-1 with full batch training. The learning rate is 0.0002. The DNN is trained by Adam optimizer (Kingma & Ba, 2014) with the MSE loss function.

In Fig. 1, for MNIST dataset, the training process of a tanh-DNN with widths 784-400-200-10 is shown in Fig. 1(a) and 1(b). For CIFAR10 dataset, results are shown in Fig. 1(c) and 1(d) of a

ReLU-CNN, which consists of one convolution layer of $3 \times 3 \times 64$, a max pooling of 2×2 , one convolution layer of $3 \times 3 \times 128$, a max pooling of 2×2 , followed by a fully-connected DNN with widths 800-400-400-400-10. For both cases, the output layer of the network is equipped with a softmax. The network output is a 10-d vector. The DNNs are trained with cross entropy loss by Adam optimizer (Kingma & Ba, 2014). (a, b) are for MNIST with a tanh-DNN. The learning rate is 0.001 with batch size 10000. After training, the training accuracy is 0.951 and test accuracy is 0.963. The amplitude of the Fourier coefficient with respect to the fourth output component at each frequency is shown in (a), in which the red dots are computed using the training data. Selected frequencies are marked by black squares. (b) $\Delta_F(k)$ at different training epochs for the selected frequencies. (c, d) are for CIFAR10 dataset. We use a ReLU network of a CNN followed by a fully-connected DNN. The learning rate is 0.003 with batch size 512. (c) and (d) are the results with respect to the ninth output component. After training, the training accuracy is 0.98 and test accuracy is 0.72.

In Fig. 2, for MNIST, we use a fully-connected tanh-DNN with widths 784-400-200-10 and MSE loss; for CIFAR10, we use cross-entropy loss and a ReLU-CNN, which consists of one convolution layer of $3 \times 3 \times 32$, a max pooling of 2×2 , one convolution layer of $3 \times 3 \times 64$, a max pooling of 2×2 , followed by a fully-connected DNN with widths 400-10 and the output layer of the network is equipped with a softmax. The learning rate for MNIST and CIFAR10 is 0.015 and 0.003, respectively. The networks are trained by Adam optimizer (Kingma & Ba, 2014) with batch size 10000. For VGG16, the learning rate is 10^{-5} . The network is trained by Adam optimizer (Kingma & Ba, 2014) with batch size 500.

In Fig. 3, the samples are evenly spaced in $[0, 1]$ with sample size 1001. We use a DNN with widths 1-4000-500-400-1 and full batch training by Adam optimizer (Kingma & Ba, 2014). The learning rate is 0.0005. β is 10. The parameters of the DNN are initialized following a Gaussian distribution with mean 0 and standard deviation 0.02.

In Fig. 4, the settings of (a) and (b) are the same as the ones in Fig. 1. For (c), we use a tanh-DNN with widths 10-500-100-1, learning rate 0.0005 under full batch-size training by Adam optimizer (Kingma & Ba, 2014). The parameters of the DNN are initialized by a Gaussian distribution with mean 0 and standard deviation 0.05.

C CENTRAL DIFFERENCE SCHEME AND JACOBI METHOD

Consider a one-dimensional (1-d) Poisson's equation:

$$\begin{aligned} -\Delta u(x) &= g(x), \quad x \in \Omega = (-1, 1) \\ u(x) &= 0, \quad x = -1, 1. \end{aligned} \quad (9)$$

$[-1, 1]$ is uniformly discretized into $n + 1$ points with grid size $h = 2/n$. The Poisson's equation in Eq. (9) can be solved by the central difference scheme,

$$-\Delta u_i = -\frac{u_{i+1} - 2u_i + u_{i-1}}{(\delta x)^2} = g(x_i), \quad i = 1, 2, \dots, n, \quad (10)$$

resulting a linear system

$$\mathbf{A}\mathbf{u} = \mathbf{g}, \quad (11)$$

where

$$\mathbf{A} = \begin{pmatrix} 2 & -1 & 0 & 0 & \cdots & 0 \\ -1 & 2 & -1 & 0 & \cdots & 0 \\ 0 & -1 & 2 & -1 & \cdots & 0 \\ \vdots & \vdots & \cdots & & & \vdots \\ 0 & 0 & \cdots & 0 & -1 & 2 \end{pmatrix}_{(n-1) \times (n-1)}, \quad (12)$$

$$\mathbf{u} = \begin{pmatrix} u_1 \\ u_2 \\ \vdots \\ u_{n-2} \\ u_{n-1} \end{pmatrix}, \quad \mathbf{g} = (\delta x)^2 \begin{pmatrix} g_1 \\ g_2 \\ \vdots \\ g_{n-2} \\ g_{n-1} \end{pmatrix}, \quad x_i = 2\frac{i}{n}. \quad (13)$$

A class of methods to solve this linear system is iterative schemes, for example, the Jacobi method. Let $\mathbf{A} = \mathbf{D} - \mathbf{L} - \mathbf{U}$, where $\mathbf{D}$ is the diagonal of $\mathbf{A}$, and $\mathbf{L}$ and $\mathbf{U}$ are the strictly lower and upper triangular parts of $-\mathbf{A}$, respectively. Then, we obtain

$$\mathbf{u} = \mathbf{D}^{-1}(\mathbf{L} + \mathbf{U})\mathbf{u} + \mathbf{D}^{-1}\mathbf{g}. \quad (14)$$

At step $t \in \mathbb{N}$, the Jacobi iteration reads as

$$\mathbf{u}^{t+1} = \mathbf{D}^{-1}(\mathbf{L} + \mathbf{U})\mathbf{u}^t + \mathbf{D}^{-1}\mathbf{g}. \quad (15)$$

We perform the standard error analysis of the above iteration process. Denote $\mathbf{u}^*$ as the true value obtained by directly performing inverse of $\mathbf{A}$ in Eq. (11). The error at step $t + 1$ is $\mathbf{e}^{t+1} = \mathbf{u}^{t+1} - \mathbf{u}^*$. Then, $\mathbf{e}^{t+1} = \mathbf{R}_J \mathbf{e}^t$, where $\mathbf{R}_J = \mathbf{D}^{-1}(\mathbf{L} + \mathbf{U})$. The converging speed of $\mathbf{e}^t$ is determined by the eigenvalues of $\mathbf{R}_J$, that is,

$$\lambda_k = \lambda_k(\mathbf{R}_J) = \cos \frac{k\pi}{n}, \quad k = 1, 2, \dots, n-1, \quad (16)$$

and the corresponding eigenvector $\mathbf{v}_k$'s entry is

$$v_{k,i} = \sin \frac{ik\pi}{n}, i = 1, 2, \dots, n-1. \quad (17)$$

So we can write

$$\mathbf{e}^t = \sum_{k=1}^{n-1} \alpha_k^t \mathbf{v}_k, \quad (18)$$

where α_k^t can be understood as the magnitude of $\mathbf{e}^t$ in the direction of $\mathbf{v}_k$. Then,

$$\begin{aligned} \mathbf{e}^{t+1} &= \sum_{k=1}^{n-1} \alpha_k^t \mathbf{R}_J \mathbf{v}_k = \sum_{k=1}^{n-1} \alpha_k^t \lambda_k \mathbf{v}_k. \\ \alpha_k^{t+1} &= \lambda_k \alpha_k^t. \end{aligned} \quad (19)$$

Therefore, the converging rate of $\mathbf{e}^t$ in the direction of $\mathbf{v}_k$ is controlled by λ_k . Since

$$\cos \frac{k\pi}{n} = -\cos \frac{(n-k)\pi}{n}, \quad (20)$$

the frequencies k and $(n-k)$ are closely related and converge with the same rate. Consider the frequency $k < n/2$, λ_k is larger for lower frequency. Therefore, lower frequency converges slower in the Jacobi method.

D PROOF OF THEOREMS

The activation function we consider is $\sigma(x) = \tanh(x)$.

$$\sigma(x) = \tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}, \quad x \in \mathbb{R}.$$

For a DNN of one hidden layer with m nodes, 1-d input x and 1-d output:

$$h(x) = \sum_{j=1}^m a_j \sigma(w_j x + b_j), \quad a_j, w_j, b_j \in \mathbb{R}, \quad (21)$$

where w_j , a_j , and b_j are called *parameters*, in particular, w_j and a_j are called *weights*, and b_j is also known as a *bias*. In the sequel, we will also use the notation $\theta = \{\theta_{lj}\}$ with $\theta_{1j} = a_j$, $\theta_{2j} = w_j$, and $\theta_{lj} = b_j$, $j = 1, \dots, m$. Note that $\hat{\sigma}(k) = -\frac{i\pi}{\sinh(\pi k/2)}$ where the Fourier transformation and its inverse transformation are defined as follows:

$$\hat{f}(k) = \int_{-\infty}^{+\infty} f(x) e^{-ikx} dx, \quad f(x) = \frac{1}{2\pi} \int_{-\infty}^{+\infty} \hat{f}(k) e^{ikx} dk.$$

The Fourier transform of $\sigma(w_j x + b_j)$ with $w_j, b_j \in \mathbb{R}, j = 1, \dots, m$ reads as

$$\sigma(\widehat{w_j \cdot + b_j})(k) = \frac{2\pi i}{|w_j|} \exp\left(\frac{ib_j k}{w_j}\right) \frac{1}{\exp(-\frac{\pi k}{2w_j}) - \exp(\frac{\pi k}{2w_j})}. \quad (22)$$

Thus

$$\hat{h}(k) = \sum_{j=1}^m \frac{2\pi a_j i}{|w_j|} \exp\left(\frac{ib_j k}{w_j}\right) \frac{1}{\exp(-\frac{\pi k}{2w_j}) - \exp(\frac{\pi k}{2w_j})}. \quad (23)$$

We define the amplitude deviation between DNN output and the *target function* $f(x)$ at frequency k as

$$D(k) \triangleq \hat{h}(k) - \hat{f}(k).$$

Write $D(k)$ as $D(k) = A(k)e^{i\phi(k)}$, where $A(k) \in [0, +\infty)$ and $\phi(k) \in \mathbb{R}$ are the amplitude and phase of $D(k)$, respectively. The loss at frequency k is $L(k) = \frac{1}{2} |D(k)|^2$, where $|\cdot|$ denotes the norm of a complex number. The total loss function is defined as: $L = \int_{-\infty}^{+\infty} L(k) dk$. Note that according to the Parseval's theorem, this loss function in the Fourier domain is equal to the commonly used loss of mean squared error, that is, $L = \int_{-\infty}^{+\infty} \frac{1}{2} (h(x) - f(x))^2 dx$. For readers' reference, we list the partial derivatives of $L(k)$ with respect to parameters

$$\frac{\partial L(k)}{\partial a_j} = \frac{2\pi}{w_j} \sin\left(\frac{b_j k}{w_j} - \phi(k)\right) E_0, \quad (24)$$

$$\begin{aligned} \frac{\partial L(k)}{\partial w_j} = & \left[\sin\left(\frac{b_j k}{w_j} - \phi(k)\right) \left(\frac{\pi^2 a_j k}{w_j^3} E_1 - \frac{2\pi a_j}{w_j^2} \right) \right. \\ & \left. - \frac{2\pi a_j b_j k}{w_j^3} \cos\left(\frac{b_j k}{w_j} - \phi(k)\right) \right] E_0, \end{aligned} \quad (25)$$

$$\frac{\partial L(k)}{\partial b_j} = \frac{2\pi a_j b_j k}{w_j^2} \cos\left(\frac{b_j k}{w_j} - \phi(k)\right) E_0, \quad (26)$$

where

$$\begin{aligned} E_0 &= \frac{\operatorname{sgn}(w_j) A(k)}{\exp(\frac{\pi k}{2w_j}) - \exp(-\frac{\pi k}{2w_j})}, \\ E_1 &= \frac{\exp(\frac{\pi k}{2w_j}) + \exp(-\frac{\pi k}{2w_j})}{\exp(\frac{\pi k}{2w_j}) - \exp(-\frac{\pi k}{2w_j})}. \end{aligned}$$

The descent increment at any direction, say, with respect to parameter θ_{lj} , is

$$\frac{\partial L}{\partial \theta_{lj}} = \int_{-\infty}^{+\infty} \frac{\partial L(k)}{\partial \theta_{lj}} dk. \quad (27)$$

The absolute contribution from frequency k to this total amount at θ_{lj} is

$$\left| \frac{\partial L(k)}{\partial \theta_{lj}} \right| \approx A(k) \exp(-|\pi k / 2w_j|) F_{lj}(\theta_j, k), \quad (28)$$

where $\theta_j \triangleq \{w_j, b_j, a_j\}$, $\theta_{lj} \in \theta_j$, $F_{lj}(\theta_j, k)$ is a function with respect to θ_j and k , which can be found in one of Eqs. (24, 25, 26).

When the component at frequency k where $\hat{h}(k)$ is not close enough to $\hat{f}(k)$, $\exp(-|\pi k / 2w_j|)$ would dominate $G_{lj}(\theta_j, k)$ for a small w_j . Through the above framework of analysis, we have the following theorem. Define

$$W = (w_1, w_2, \dots, w_m)^T \in \mathbb{R}^m. \quad (29)$$

Theorem. Consider a one hidden layer DNN with activation function $\sigma(x) = \tanh x$. For any frequencies k_1 and k_2 such that $|\hat{f}(k_1)| > 0$, $|\hat{f}(k_2)| > 0$, and $|k_2| > |k_1| > 0$, there exist positive constants c and C such that for sufficiently small δ , we have

$$\frac{\mu\left(\left\{W : \left|\frac{\partial L(k_1)}{\partial \theta_{lj}}\right| > \left|\frac{\partial L(k_2)}{\partial \theta_{lj}}\right| \text{ for all } l, j\right\} \cap B_\delta\right)}{\mu(B_\delta)} \geq 1 - C \exp(-c/\delta), \quad (30)$$

where $B_\delta \subset \mathbb{R}^m$ is a ball with radius δ centered at the origin and $\mu(\cdot)$ is the Lebesgue measure.

We remark that c and C depend on $k_1, k_2, |\hat{f}(k_1)|, |\hat{f}(k_2)|, \sup |a_i|, \sup |b_i|$, and m .

Proof. To prove the statement, it is sufficient to show that $\mu(S_{l_j, \delta})/\mu(B_\delta) \leq C \exp(-c/\delta)$ for each l, j , where

$$S_{l_j, \delta} := \left\{ W \in B_\delta : \left| \frac{\partial L(k_1)}{\partial \theta_{l_j}} \right| \leq \left| \frac{\partial L(k_2)}{\partial \theta_{l_j}} \right| \right\}. \quad (31)$$

We prove this for $S_{1j, \delta}$, that is, $\theta_{l_j} = a_j$. The proofs for $\theta_{l_j} = w_j$ and b_j are similar. Without loss of generality, we assume that $k_1, k_2 > 0, b_j > 0$, and $w_j \neq 0, j = 1, \dots, m$. According to Eq. (24), the inequality $|\frac{\partial L(k_1)}{\partial a_j}| \leq |\frac{\partial L(k_2)}{\partial a_j}|$ is equivalent to

$$\frac{A(k_2)}{A(k_1)} \left| \frac{\exp(\frac{\pi k_1}{2w_j}) - \exp(-\frac{\pi k_1}{2w_j})}{\exp(\frac{\pi k_2}{2w_j}) - \exp(-\frac{\pi k_2}{2w_j})} \right| \cdot \left| \sin\left(\frac{b_j k_2}{w_j} - \phi(k_2)\right) \right| \geq \left| \sin\left(\frac{b_j k_1}{w_j} - \phi(k_1)\right) \right| \quad (32)$$

Note that $|\hat{h}(k)| \leq C \sum_{j=1}^m \frac{|a_j|}{|w_j|} \exp(-\frac{\pi k}{2|w_j|})$ for $k > 0$. Thus

$$\lim_{W \rightarrow 0} \hat{h}(k) = 0 \quad \text{and} \quad \lim_{W \rightarrow 0} D(k) = -\hat{f}(k). \quad (33)$$

Therefore,

$$\lim_{W \rightarrow 0} A(k) = |\hat{f}(k)| \quad \text{and} \quad \lim_{W \rightarrow 0} \phi(k) = \pi + \arg(\hat{f}(k)). \quad (34)$$

For $W \in B_\delta$ with sufficiently small δ , $A(k_1) > \frac{1}{2}|\hat{f}(k_1)| > 0$ and $A(k_2) < 2|\hat{f}(k_2)|$. Also note that $|\sin(\frac{b_j k_2}{w_j} - \phi(k_2))| \leq 1$ and that for sufficiently small δ ,

$$\left| \frac{\exp(\frac{\pi k_1}{2w_j}) - \exp(-\frac{\pi k_1}{2w_j})}{\exp(\frac{\pi k_2}{2w_j}) - \exp(-\frac{\pi k_2}{2w_j})} \right| \leq 2 \exp\left(-\frac{\pi(k_2 - k_1)}{2|w_j|}\right). \quad (35)$$

Thus, inequality (32) implies that

$$\left| \sin\left(\frac{b_j k_1}{w_j} - \phi(k_1)\right) \right| \leq \frac{8|\hat{f}(k_2)|}{|\hat{f}(k_1)|} \exp\left(-\frac{\pi(k_2 - k_1)}{2|w_j|}\right). \quad (36)$$

Noticing that $\frac{2}{\pi}|x| \leq |\sin x|$ ($|x| \leq \frac{\pi}{2}$) and Eq. (34), we have for $W \in S_{l_j, \delta}$, for some $q \in \mathbb{Z}$,

$$\left| \frac{b_j k_1}{w_i} - \arg(\hat{f}(k_1)) - q\pi \right| \leq \frac{8\pi|\hat{f}(k_2)|}{|\hat{f}(k_1)|} \exp\left(-\frac{\pi(k_2 - k_1)}{2\delta}\right) \quad (37)$$

that is,

$$-c_1 \exp(-c_2/\delta) + q\pi + \arg(\hat{f}(k_1)) \leq \frac{b_j k_1}{w_i} \leq c_1 \exp(-c_2/\delta) + q\pi + \arg(\hat{f}(k_1)), \quad (38)$$

where $c_1 = \frac{8\pi|\hat{f}(k_2)|}{|\hat{f}(k_1)|}$ and $c_2 = \pi(k_2 - k_1)$. Define $I := I^+ \cup I^-$ where

$$I^+ := \{w_j > 0 : W \in S_{1j, \delta}\}, \quad I^- := \{w_j < 0 : W \in S_{1j, \delta}\}. \quad (39)$$

For $w_j > 0$, we have for some $q \in \mathbb{Z}$,

$$0 < \frac{b_j k_1}{c_1 \exp(-c_2/\delta) + q\pi + \arg(\hat{f}(k_1))} \leq w_j \leq \frac{b_j k_1}{-c_1 \exp(-c_2/\delta) + q\pi + \arg(\hat{f}(k_1))}. \quad (40)$$

Since $W \in B_\delta$ and $c_1 \exp(-c_2/\delta) + \arg(\hat{f}(k_1)) \leq 2\pi$, we have $\frac{b_j k_1}{2\pi + q\pi} \leq w_j \leq \delta$. Then Eq. (40) only holds for some large q , more precisely, $q \geq q_0 := \frac{b_j k}{\pi\delta} - 2$. Thus we obtain the estimate for the (one-dimensional) Lebesgue measure of I^+

$$\begin{aligned} \mu(I^+) &\leq \sum_{q=q_0}^{\infty} \left| \frac{b_j k_1}{-c_1 \exp(-c_2/\delta) + q\pi + \arg(\hat{f}(k_1))} - \frac{b_j k_1}{c_1 \exp(-c_2/\delta) + q\pi + \arg(\hat{f}(k_1))} \right| \\ &\leq 2|b_j|k_1 c_1 \exp(-c_2/\delta) \cdot \sum_{q=q_0}^{\infty} \frac{1}{(q\pi + \arg(\hat{f}(k_1)))^2 - (c_1 \exp(-c_2/\delta))^2} \\ &\leq C \exp(-c/\delta). \end{aligned} \quad (41)$$

The similar estimate holds for $\mu(I^-)$, and hence $\mu(I) \leq C \exp(-c/\delta)$. For $W \in B_\delta$, the $(m-1)$ dimensional vector $(w_1, \dots, w_{j-1}, w_{j+1}, \dots, w_m)^T$ is in a ball with radius δ in $\mathbb{R}^{m-1}$. Therefore, we final arrive at the desired estimate

$$\frac{\mu(S_{1j,\delta})}{\mu(B_\delta)} \leq \frac{\mu(I)\omega_{m-1}\delta^{m-1}}{\omega_m\delta^m} \leq C \exp(-c/\delta), \quad (42)$$

where ω_m is the volume of a unit ball in $\mathbb{R}^m$. $\square$

Theorem. *Considering a DNN of one hidden layer with activation function $\sigma(x) = \tanh(x)$. Suppose the target function has only two non-zero frequencies k_1 and k_2 , that is, $|\hat{f}(k_1)| > 0$, $|\hat{f}(k_2)| > 0$, and $|k_2| > |k_1| > 0$, and $|\hat{f}(k)| = 0$ for $k \neq k_1, k_2$. Consider the loss function of $L = L(k_1) + L(k_2)$ with gradient descent training. Denote*

$$\mathcal{S} = \left\{ \frac{\partial L(k_1)}{\partial t} \leq 0, \frac{\partial L(k_1)}{\partial t} \leq \frac{\partial L(k_2)}{\partial t} \right\},$$

that is, $L(k_1)$ decreases faster than $L(k_2)$. There exist positive constants c and C such that for sufficiently small δ , we have

$$\frac{\mu(\{W : \mathcal{S} \text{ holds}\} \cap B_\delta)}{\mu(B_\delta)} \geq 1 - C \exp(-c/\delta),$$

where $B_\delta \subset \mathbb{R}^m$ is a ball with radius δ centered at the origin and $\mu(\cdot)$ is the Lebesgue measure.

Proof. By gradient descent algorithm, we obtain

$$\begin{aligned} \frac{\partial L(k_1)}{\partial t} &= \sum_{l,j} \frac{\partial L(k_1)}{\partial \theta_{lj}} \frac{\partial \theta_{lj}}{\partial t} \\ &= - \sum_{l,j} \frac{\partial L(k_1)}{\partial \theta_{lj}} \frac{\partial (L(k_1) + L(k_2))}{\partial \theta_{lj}} \\ &= - \sum_{l,j} \left(\frac{\partial L(k_1)}{\partial \theta_{lj}} \right)^2 - \sum_{l,j} \frac{\partial L(k_1)}{\partial \theta_{lj}} \frac{\partial L(k_2)}{\partial \theta_{lj}}, \\ \frac{\partial L(k_2)}{\partial t} &= - \sum_{l,j} \left(\frac{\partial L(k_2)}{\partial \theta_{lj}} \right)^2 - \sum_{l,j} \frac{\partial L(k_1)}{\partial \theta_{lj}} \frac{\partial L(k_2)}{\partial \theta_{lj}}, \end{aligned}$$

and

$$\frac{\partial L}{\partial t} = \frac{\partial (L(k_1) + L(k_2))}{\partial t} = - \sum_{l,j} \left(\frac{\partial L(k_1)}{\partial \theta_{lj}} + \frac{\partial L(k_2)}{\partial \theta_{lj}} \right)^2 \leq 0. \quad (43)$$

To obtain

$$0 < \frac{\partial L(k_1)}{\partial t} - \frac{\partial L(k_2)}{\partial t} = - \sum_{l,j} \left[\left(\frac{\partial L(k_1)}{\partial \theta_{lj}} \right)^2 - \left(\frac{\partial L(k_2)}{\partial \theta_{lj}} \right)^2 \right], \quad (44)$$

it is sufficient to have

$$\left| \frac{\partial L(k_1)}{\partial \theta_{lj}} \right| > \left| \frac{\partial L(k_2)}{\partial \theta_{lj}} \right|. \quad (45)$$

Eqs. (43, 44) also yield to

$$\frac{\partial L(k_1)}{\partial t} < 0.$$

Therefore, Eq. (45) is a sufficient condition for $\mathcal{S}$. Based on the theorem 1, we have proved the theorem 2. $\square$

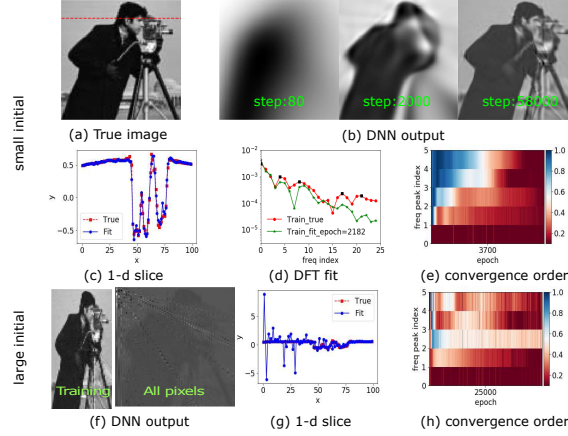


Figure 6: F-Principle in fitting a natural image. The training data are all pixels whose horizontal indices are odd. We initialize DNN parameters by a Gaussian distribution with mean 0 and standard deviation 0.08 (small initial) or 1 (large initial). (a) True image. (b-g) correspond to the case of the small initial parameters. (f-h) correspond to the case of the large initial parameters. (b) DNN outputs of all pixels at different training epochs. (c, g) DNN outputs (blue) and the true gray-scale (red) of test pixels at the red dashed position in (a). (d) $|\hat{h}(k)|$ (green) at certain training epoch and $|\hat{f}(k)|$ (red) at the red dashed position in (a), as a function of frequency index. Selected peaks are marked by black dots. (e, h) $\Delta_F(k)$ computed by the training data at different epochs for the selected frequencies in (d). (f) DNN outputs of training pixels (left) and all pixels (right) after training. We use a tanh-DNN with widths 2-400-200-100-1. We train the DNN with the full batch and learning rate 0.0002. The DNN is trained by Adam optimizer (Kingma & Ba, 2014) with the MSE loss function.

E MEMORIZING 2-D IMAGE

We train a DNN to fit a natural image (See Fig. 6(a)), a mapping from coordinate (x, y) to gray scale strength, where the latter is subtracted by its mean and then normalized by the maximal absolute value. First, we initialize DNN parameters by a Gaussian distribution with mean 0 and standard deviation 0.08 (initialization with small parameters). From the snapshots during the training process, we can see that the DNN captures the image from coarse-grained low frequencies to detailed high frequencies (Fig. 6(b)). As an illustration of the F-Principle, we study the Fourier transform of the image with respect to x for a fixed y (red dashed line in Fig. 6(a), denoted as the target function $f(x)$ in the spatial domain). The DNN can well capture this 1-d slice after training as shown in Fig. 6(c). Fig. 6(d) displays the amplitudes $|\hat{f}(k)|$ of the first 40 frequency components. Due to the small initial parameters, as an example in Fig. 6(d), when the DNN is fitting low-frequency components, high frequencies stay relatively small. As the relative error shown in Fig. 6(e), the first five frequency peaks converge from low to high in order.

Next, we initialize DNN parameters by a Gaussian distribution with mean 0 and standard deviation 1 (initialization with large parameters). After training, the DNN can well capture the training data, as shown in the left in Fig. 6(f). However, the DNN output at the test pixels are very noisy, as shown in the right in Fig. 6(f). For the pixels at the red dashed lines in Fig. 6(a), as shown in Fig. 6(g), the DNN output fluctuates a lot. Compared with the case of small initial parameters, as shown in Fig. 6(h), the convergence order of the first five frequency peaks do not have a clear order.

F ANOTHER VIEWPOINT OF EXAMINING F-PRINCIPLE IN MNIST/CIFAR10 THROUGH FILTERING METHOD

The section we present another viewpoint of examining F-Principle in MNIST/CIFAR10 through filtering method. For readers' convenience, we describe the filtering method again in this section.

We decompose the frequency space into two domains by a constant k_0 , i.e., a low-frequency domain of $|\mathbf{k}| \leq k_0$ and a high-frequency domain of $|\mathbf{k}| > k_0$, where $|\cdot|$ is the length of a vector. Then, $\mathbf{y}_i$ can be decomposed by $\mathbf{y}_i = \mathbf{y}_i^{\text{low},k_0} + \mathbf{y}_i^{\text{high},k_0}$, where $\mathbf{y}_i^{\text{low},k_0}$ and $\mathbf{y}_i^{\text{high},k_0}$ are the low- and high-frequency part of $\mathbf{y}_i$, respectively. For illustration, $y_i = \sin(2\pi x_i) + \sin(2\pi(2x_i)) + \sin(3\pi(3x_i))$, if $k_0 = 1.5$, then, $y_i^{\text{low},k_0} = \sin(2\pi x_i)$ and $y_i^{\text{high},k_0} = \sin(2\pi(2x_i)) + \sin(3\pi(3x_i))$.

The DNN is trained as usual by the original dataset $\{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=0}^{n-1}$. During the training, we examine the distance between the DNN output and the low-frequency part of $\{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=0}^{n-1}$ by MSE, i.e., $\text{Dist}(\mathbf{y}^{\text{low},k_0}, \mathbf{h})$. Under the F-Principle, the DNN training would be dominated by the low-frequency part $\mathbf{y}^{\text{low},k_0}$ at the early stage, therefore, $\text{Dist}(\mathbf{y}^{\text{low},k_0}, \mathbf{h})$ would decrease. At the latter stage, the training would be dominated by high-frequency part $\mathbf{y}_i^{\text{high},k_0}$, which would make the the DNN output deviate from the low-frequency part $\mathbf{y}^{\text{low},k_0}$, therefore, $\text{Dist}(\mathbf{y}^{\text{low},k_0}, \mathbf{h})$ would increase. In short, F-Principle predicts that during the training of original dataset $\{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=0}^{n-1}$, $\text{Dist}(\mathbf{y}^{\text{low},k_0}, \mathbf{h})$ would first decrease and then increase.

We refer to the *turning epoch of training* when $\text{Dist}(\mathbf{y}^{\text{low},k_0}, \mathbf{h})$ attains its minimum, denoted by T_{k_0} . If $k_1 > k_0$, then, $\mathbf{y}^{\text{low},k_1}$ preserves not only all frequency components in $\mathbf{y}^{\text{low},k_0}$ but also those between k_0 and k_1 . The DNN would spend more time to converge all frequency components of $\mathbf{y}^{\text{low},k_1}$ compared with $\mathbf{y}^{\text{low},k_0}$. Therefore, $T_{k_1} > T_{k_0}$. In short, F-Principle predicts that during the training of original dataset $\{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=0}^{n-1}$, T_{k_0} monotonically increases with k_0 .

Note that $\text{Dist}(\mathbf{y}^{\text{low},k_0}, \mathbf{h})$ is *not* a generalization error *nor* a test error because $\text{Dist}(\mathbf{y}^{\text{low},k_0}, \mathbf{h})$ quantifies how well DNN learns the low frequency part of the training dataset. $\mathbf{y}^{\text{low},k_0}$ can be obtained by the following filtering method.

F.1 FILTERING METHOD

We obtain the low-frequency part $\mathbf{y}^{\text{low},k_0}$ by convolving the original dataset $\{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=0}^{n-1}$ with a Gaussian filter. An intuition of why such operation can eliminate the high-frequency part of the original dataset is as follows. The convolution in the spatial domain is equivalent to the product in the frequency domain. The Fourier transform of a Gaussian kernel is still a Gaussian kernel. Therefore, the convolution result is the product of a Gaussian kernel with the Fourier transform of the original dataset. Since the tail of a Gaussian kernel exponentially decays, high-frequency components after filtering almost vanish due to the product with small numbers (close to zero).

We train the DNN with *original dataset* $\{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=0}^{n-1}$. The *Gaussian-filtered dataset* $\{(\mathbf{x}_i, \mathbf{y}_i^\delta)\}_{i=0}^{n-1}$ used to examine the DNN at each training epoch can be obtained by

$$\mathbf{y}_i^\delta = \frac{1}{C_i} \sum_{j=0}^{n-1} \mathbf{y}_j \exp(-|\mathbf{x}_i - \mathbf{x}_j|^2 / (2\delta)),$$

where $C_i = \sum_{j=0}^{n-1} \exp(-|\mathbf{x}_i - \mathbf{x}_j|^2 / (2\delta))$. For fixed δ , clearly, $\{\mathbf{y}_i^\delta\}$ preserves the low frequency part while losing the high frequency part of $\{\mathbf{y}_i\}$. When $\delta \rightarrow 0$, $\mathbf{y}_i^\delta \rightarrow \mathbf{y}_i$, i.e., keeping all frequencies. When $\delta \rightarrow \infty$, $\mathbf{y}_i^\delta \rightarrow \frac{1}{n} \sum_{j=0}^{n-1} \mathbf{y}_j$, i.e., keeping only the lowest (zero) frequency. As δ increases, $\{\mathbf{y}_i^\delta\}$ preserves less low-frequency components. The turning epoch, T_δ , would then decrease with δ . Therefore, the F-Principle predicts that *during the training of original dataset* $\{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=0}^{n-1}$:

First, for a fixed δ , the distance between the DNN output and a low-frequency part of $\{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=0}^{n-1}$, $D(\mathbf{y}^\delta, \mathbf{h})$, would first decrease and then increase.

Second, T_δ monotonically decreases with δ .

Note that if an algorithm captures the target function from high to low frequency, these two predictions fail. Ideal experiments are shown in Appendix G to illustrate this point.

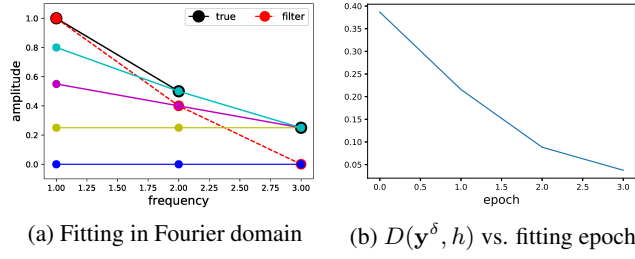


Figure 7: Ideal experiment. In (a), the blue, yellow, magenta and cyan curves correspond to fitting curves at the initial, first, second and third fitting epoch.

F.2 DNNs WITH VARIOUS SETTINGS

With the filtering method, we show the F-Principle in the DNN training process of real datasets. For MNIST, we use a fully-connected tanh-DNN (no softmax) with MSE loss; for CIFAR10, we use cross-entropy loss and a ReLU-CNN, followed by a fully-connected DNN with a softmax.

As an example, results of each dataset for one δ are shown in Fig. 8(a). In both cases, $D(\mathbf{y}^\delta, \mathbf{h})$ first decreases and then increases, which meet the first prediction.

As shown in Fig. 8(b), T_δ monotonically decreases with δ , which meets the second prediction. We also remark that, based on above results on cross-entropy loss, the F-Principle is not limited to MSE loss, which possesses a natural Fourier domain interpretation by the Parseval’s theorem as illustrated in (Xu et al., 2018) and (Rahaman et al., 2018). Note that the above results holds for optimization methods of both gradient descent and stochastic gradient descent.

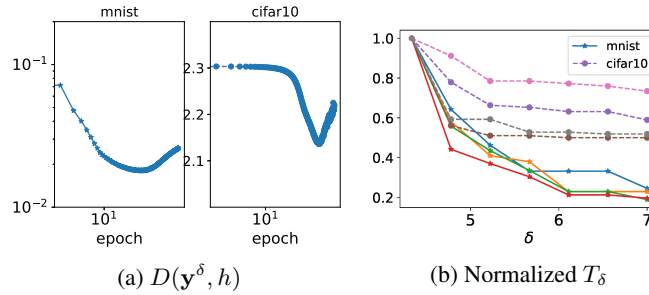


Figure 8: F-Principle in MNIST and CIFAR10. (a) $D(\mathbf{y}^\delta, \mathbf{h})$ against training epoch. $\delta = 7$ and 4.3 for MNIST (left) and CIFAR10 (right), respectively. (b) T_δ , normalized by the maximal T_δ of each trial, is plotted against filter width δ . Solid and dashed curves are for MNIST and CIFAR10, respectively. Each curve is for one trial.

F.3 F-PRINCIPLE IN VGG16

It is important to verify the F-Principle in a commonly used and large DNNs. Therefore, we use the filtering method to show the F-Principle in the VGG16 (Simonyan & Zisserman, 2014) equipped with a 1024 fully-connected layer. We train the network with CIFAR10 from *scratch*. As shown in Fig. 9 (a) and (b), the phenomena are consistent with the first and second prediction in Section F.1.

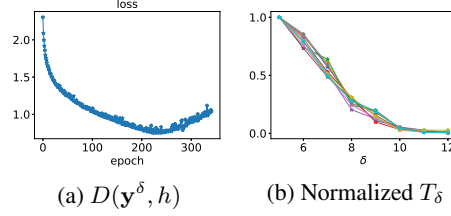


Figure 9: F-Principle in VGG16 with CIFAR10. (a) $D(\mathbf{y}^\delta, h)$ against training epoch. $\delta = 4$. (b) T_δ , normalized by the maximal T_δ of each trial, is plotted against filter width δ . Each curve is for one trial.

G BEHAVIOR OF ANTI-F-PRINCIPLE IN SYNTHETIC DATA THROUGH FILTERING METHOD

G.1 EXPERIMENTS

Consider a target function is

$$f(x) = c_0 + \sum_{k=1} c_k \sin(2k-1)x.$$

The fitting

$$h(x, t) = c_0(1 - \exp(-a_0 t)) + \sum_{k=1} (1 - \exp(-a_k t)) c_k \sin(2k-1)x.$$

In this section, we refer to F-Principle (anti-F-Principle) if a_k monotonically decreases (increases) as k , i.e., low (high) frequency has higher priority when $h(x, t)$ converges to $f(x)$ as $t \rightarrow \infty$. Fig. 10 shows that F-Principle and anti-F-Principle have different behavior in the filtering method. In any case, the F-Principle meets the two predictions in the main text, that is, *during the training of original dataset* $\{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=0}^{n-1}$:

First, for a fixed δ , the distance between the DNN output and a low-frequency part of $\{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=0}^{n-1}$, $D(\mathbf{y}^\delta, h)$, would first decrease and then increase.

Second, T_δ monotonically decreases with δ .

An intuitive understanding of the anti-F-Principle is in the next sub-section.

G.2 UNDERSTANDING OF THE PHENOMENON OF ANTI-F-PRINCIPLE

If an algorithm captures the target function from high to low frequency, the two F-Principle predictions fail. Here show ideal experiments in Fig. 11 to illustrate this claim.

Firstly, consider that the target function decays in Fourier domain. In Fig. 11a, the target function has three frequencies, marked by black dots; the filtered data $\mathbf{y}^\delta$, marked by red dots and dashed line, preserves all of the first frequency, most of the second frequency, and non the third frequency. The initial value is zero at all three frequencies (Note that the initial DNN output is often close to zero). As high frequency converges faster, the third frequency converges while other two frequencies do not, denoted by the yellow curve. Despite the fitting curve deviates from the filtered data at the third frequency, due to the large amplitude of first two frequencies, the combined contribution from the first two frequencies leads the fitting curve closer to the filtered data, i.e., $D(\mathbf{y}^\delta, h)$ decreases. As the fitting curve evolves from the magenta one to the cyan one, the fitting curve deviates from the filtered data at the second frequency, however, it gets much closer to the filtered data at the first frequency, leading to decrement of $D(\mathbf{y}^\delta, h)$. Therefore, $D(\mathbf{y}^\delta, h)$ decreases during the training (Fig. 11b), i.e., contradicting to the F-Principle's predictions fail.

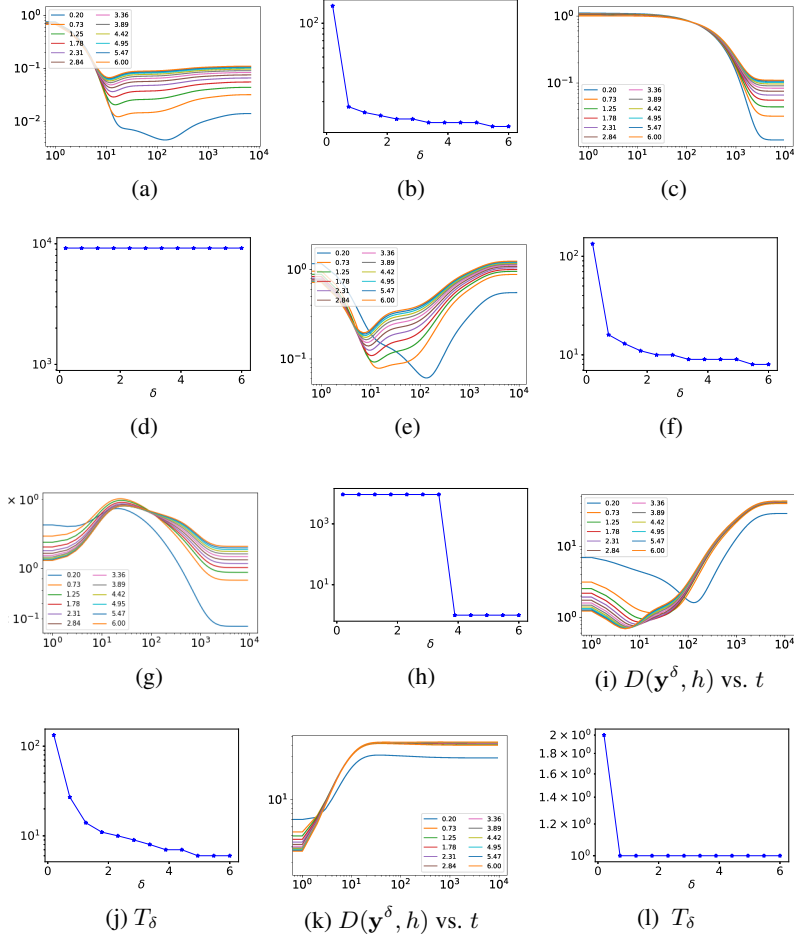


Figure 10: The priority coefficient $a_k \in [200, 150, 5, 1]$. Left two columns (F-Principle): lower frequency has higher priority; right two columns (anti-F-Principle): higher frequency has higher priority. The amplitude for each frequency, c_k , from low to high is $[1, 1/2, 1/5, 1/8]$, $[1, 1, 1, 1]$, $[1, 2, 5, 8]$ for three rows, respectively. In the first and the third column, the legend indicates δ . x is evenly sampled from $[-6.28, 6.28]$ with size 100. The output and the loss are computed every $dt = 0.001$.

Secondly, consider that the target function keeps constant in Fourier domain. At a early stage, the fitting curve evolves from the blue solid one to the blue dashed one, the summation of the first two frequencies, which makes the fitting curve closer to the filtered one (red dashed), is large than the change of the third frequency, then, $D(\mathbf{y}^\delta, h)$ decreases. At the second stage, the fitting curve evolves from the magenta solid one to the magenta dashed one, the summation of the second and the third frequencies, which makes the fitting curve deviate from the filtered one (red dashed), is large than the change of the first frequency, then, $D(\mathbf{y}^\delta, h)$ increases. At final stage, only the first frequency, which is shared with the filtered data, does not converge yet. Then, converging the first frequency makes $D(\mathbf{y}^\delta, h)$ decreases. Therefore, $D(\mathbf{y}^\delta, h)$ first decreases, then increases, and finally decreases during the training (Fig. 11d), i.e., contradicting to the F-Principle's predictions fail.

Thirdly, consider that the target function increases in Fourier domain. Before the final stage, the third frequency dominates the evolution (Fig. 11e), thus, $D(\mathbf{y}^\delta, h)$ increases. At final stage, $D(\mathbf{y}^\delta, h)$ increases slowly due to the converging of the first frequency, which is small-amplitude. Therefore, $D(\mathbf{y}^\delta, h)$ almost always increases during the training (Fig. 11f), i.e., contradicting to the F-Principle's predictions fail.

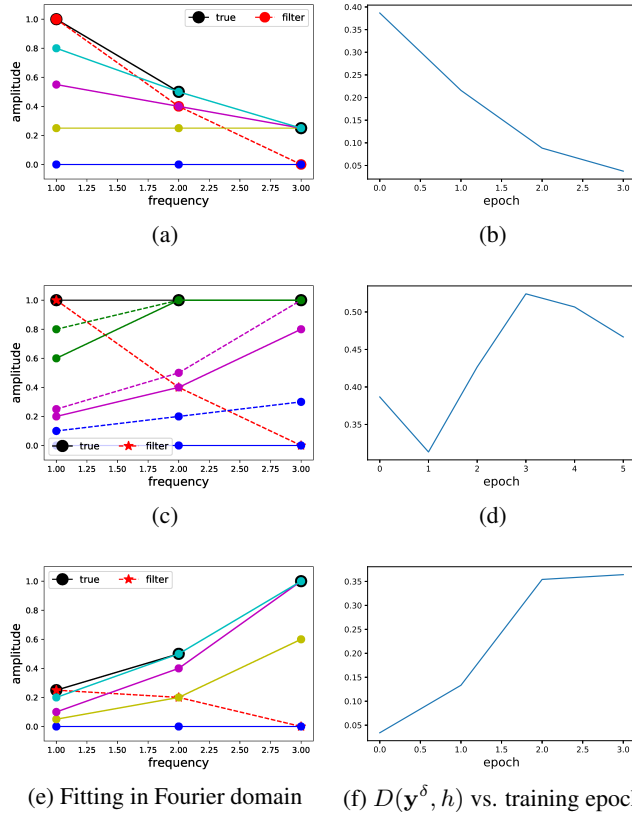


Figure 11: Ideal experiment. Each row is for one target function, indicated by black dots. In the first column, the curves with legends are fitting curve at certain training epoch. A lower one is from a earlier training epoch.